

# Cấu trúc dữ liệu: Set, Map, Multiset

## 1. Tại sao cần set và map?

Hãy xem bài toán: Cho dãy  $n$  số. Với mỗi truy vấn, hỏi xem một số  $x$  có trong dãy không.

Nếu dùng mảng:

- Tìm kiếm tuyến tính:  $O(n)$  mỗi truy vấn. Với  $q = 10^5, n = 10^5 \rightarrow 10^{10}$  phép tính  $\rightarrow$  TLE!
- Sắp xếp + binary search:  $O(n \log n + q \log n)$  – tốt, nhưng nếu có thêm thao tác thêm/xóa phần tử xen kẽ truy vấn thì sao? Mảng sẽ bị chậm khi chèn/xóa ( $O(n)$ ).

Cách dùng set: Đặt tất cả số vào set, kiểm tra trong  $O(\log n)$  – tổng  $O((n + q) \log n) \approx 2 \times 10^6 \rightarrow$  AC!

Đó là một trong rất nhiều ví dụ. set và map giúp:

- Kiểm tra phần tử có tồn tại không
- Đếm số phần tử khác nhau
- Tìm phần tử lớn nhất/nhỏ nhất  $\geq$  hoặc  $> x$
- Lưu key-value mà key tự động được sắp xếp

## 2. `std::set` – tập hợp các giá trị khác nhau, đã sắp xếp

set lưu các phần tử khác nhau từng đôi, tự động sắp xếp tăng dần.

Lưu ý cho người dùng Python: Python không có cấu trúc tương đương `std::set` / `std::map` đã sắp xếp trong thư viện chuẩn. `set` / `dict` của Python là bảng băm –  $O(1)$  trung bình nhưng không có thứ tự, không có `lower_bound`. Đối với các bài cần tập/map có thứ tự (đây là lý do chính của bài học này), hãy dùng C++. Toàn bộ ví dụ trong bài này được viết bằng C++.

### Khai báo và thao tác cơ bản

C++

```
set<int> s;

s.insert(5);    // O(log n)
s.insert(3);
s.insert(5);    // Không thêm nữa, vì 5 đã có

s.erase(3);     // Xóa giá trị 3, O(log n)
```

```

if (s.count(5)) {} // Kiểm tra có tồn tại không,  $O(\log n)$ 

cout << s.size(); // Số phần tử

for (int x : s) cout << x << ' '; // Duyệt theo thứ tự tăng dần

```

Thời gian các thao tác chính: insert, erase, find, count đều  $O(\log n)$ .

## Tìm phần tử lớn nhất / nhỏ nhất

C++

```

int mn = *s.begin(); // Nhỏ nhất
int mx = *s.rbegin(); // Lớn nhất (rbegin = reverse begin)

```

## lower\_bound và upper\_bound – siêu mạnh!

C++

- `s.lower_bound(x)`: trả về phần tử nhỏ nhất  $\geq x$
- `s.upper_bound(x)`: trả về phần tử nhỏ nhất  $> x$

```

auto it = s.lower_bound(7);
if (it != s.end()) cout << *it; // Số nhỏ nhất  $\geq 7$ 

```

Lưu ý: Phải dùng `s.lower_bound(x)`, không phải `lower_bound(s.begin(), s.end(), x)` – cái sau là  $O(n)$  vì set không phải mảng truy cập ngẫu nhiên!

## Ví dụ: Tìm số lớn nhất nhỏ hơn x

C++

```

auto it = s.lower_bound(x);
if (it != s.begin()) {
    --it;
    cout << *it; // Số lớn nhất < x
}

```

## 3. std::multiset – cho phép trùng

multiset giống set nhưng cho phép phần tử trùng nhau.

C++

```

multiset<int> ms;
ms.insert(5);
ms.insert(5); // OK, co 2 phan tu 5
ms.insert(3);

```

```

cout << ms.count(5); // 2

```

```

ms.erase(ms.find(5)); // Xoa MOT phan tu 5 (quan trong!)
// ms.erase(5) se xoa TAT CA phan tu 5 -- can than!

```

Bây thường gặp: `ms.erase(value)` xóa tất cả phần tử bằng `value`. Để xóa chỉ một, dùng `ms.erase(ms.find(value))`.

## 4. `std::map` – ánh xạ `key` $\rightarrow$ `value`

`map` lưu các cặp (`key`, `value`), `key` tự động sắp xếp.

C++

```

map<string, int> cnt;

cnt["apple"] = 3;
cnt["banana"] = 5;
cnt["apple"]++; // Gio apple = 4

if (cnt.count("apple")) {} // Kiem tra key ton tai

for (auto [k, v] : cnt) { // C++17, duyet theo thu tu key
    cout << k << ' ' << v << '\n';
}

```

Bây: `cnt[key]` mà `key` chưa tồn tại sẽ tự động tạo với giá trị mặc định (0 cho `int`). Nếu chỉ muốn kiểm tra, dùng `cnt.count(key)` hoặc `cnt.find(key)`.

## Đếm tần suất (cực kỳ phổ biến!)

C++

```

map<int, int> freq;
for (int x : a) freq[x]++;

// So khác nhau
cout << freq.size();

int mx = 0;
for (auto [k, v] : freq) mx = max(mx, v);

```

## 5. unordered\_set / unordered\_map – bảng băm (hash)

- set / map: cây cân bằng (Red-Black Tree),  $O(\log n)$  có thứ tự
- unordered\_set / unordered\_map: bảng băm,  $O(1)$  trung bình, không có thứ tự

C++

```
unordered_map<int, int> cnt;  
cnt[x]++; // O(1) trung bình
```

Khi nào dùng cái nào?

- Cần thứ tự (lower\_bound, duyệt tăng dần)  $\rightarrow$  set / map
- Không cần thứ tự, chỉ cần tra cứu nhanh  $\rightarrow$  unordered\_set / unordered\_map

Cảnh báo: unordered\_map có thể bị anti-hash attack (TLE do chủ ý). Trong CP an toàn nhất là dùng map trừ khi thực sự cần  $O(1)$ .

## 6. Mẫu bài toán điển hình

### Mẫu 1: Số khác nhau trong mảng

C++

```
set<int> s(a.begin(), a.end());  
cout << s.size();
```

### Mẫu 2: Có cặp $(i, j)$ với $a_i + a_j = T$ không?

Duyệt  $i$ , kiểm tra  $T - a_i$  trong set:

C++

```
set<int> seen;  
for (int x : a) {  
    if (seen.count(T - x)) { /* tìm thấy */ }  
    seen.insert(x);  
}
```

### Mẫu 3: Online – sau mỗi lần thêm 1 số, hỏi số nhỏ thứ $k$

Với  $k = 1$  (nhỏ nhất): chỉ cần `*ms.begin()`.

Với  $k$  bất kỳ: multiset không hỗ trợ truy cập theo chỉ số trong  $O(\log n)$ . Cần dùng Policy-based data structure (`__gnu_pbds::tree`).

## Mẫu 4: Đếm nghịch thế (số cặp $i < j$ với $a_i > a_j$ )

Bài toán này cần đếm nhanh số phần tử lớn hơn  $a_j$  trong tập đã thấy. set / map thường không hỗ trợ trực tiếp ( $O(n)$ ). Cần Fenwick tree hoặc Policy-based tree (`__gnu_pbds`) sẽ học ở khóa Vàng.

## Mẫu 5: Hai cấu trúc song song – duy trì cả phần tử và "khoảng cách" giữa các phần tử

Một số bài cần không chỉ tập các giá trị, mà còn các khoảng trống/khoảng cách giữa các giá trị liên kề. Mẹo: duy trì hai cấu trúc song song:

- `set<int> S` – chứa các giá trị hiện có (sắp xếp).
- `multiset<int> gaps` – chứa các khoảng cách giữa hai phần tử kề nhau trong `S`.

Khi thêm/xóa  $x$  vào  $S$ :

1. Tìm hai phần tử  $l, r$  kề  $x$  trong  $S$ .
2. Xóa khoảng cũ  $r - l$  khỏi `gaps`.
3. Thêm hai khoảng mới  $x - l$  và  $r - x$  vào `gaps`.
4. Cập nhật  $S$ .

Khoảng dài nhất hiện tại = `*gaps.rbegin()` trong  $O(1)$ .

**C++**

```
set<int> S = {0, L}; // hai biên
multiset<int> gaps = {L};

void addPoint(int x) {
    auto it = S.lower_bound(x);
    int r = *it, l = *prev(it);

    gaps.erase(gaps.find(r - l)); // xóa MOT khoảng cũ
    gaps.insert(x - l);
    gaps.insert(r - x);

    S.insert(x);
}

int longestGap() { return *gaps.rbegin(); } // Lon nhất
```

Bài luyện tập kinh điển là CSES Traffic Lights (`cses1163`).

## 7. Bẫy thường gặp & Mẹo

- set không có random access: không có `s[i]`. Phải dùng iterator hoặc `next(s.begin(), i)` – và cái sau là  $O(i)$ !
- Iterator bị invalid khi erase: lưu lại trước hoặc dùng `it = s.erase(it)`.
- Hằng số lớn: set / map chậm hơn array khoảng 5-10 lần. Nếu  $n$  lớn ( $\geq 10^6$ ) và chỉ cần kiểm tra tồn tại trong tập hợp số nhỏ, hãy dùng `vector<bool>` hoặc `bitset`.
- `map[key]` tạo entry: nếu chỉ tra cứu, dùng `find` hoặc `count`.

## 8. Tổng kết

| CẤU TRÚC          | CHO PHÉP TRÙNG  | CÓ THỨ TỰ | THAO TÁC    |
|-------------------|-----------------|-----------|-------------|
| set               | Không           | Có        | $O(\log n)$ |
| multiset          | Có              | Có        | $O(\log n)$ |
| map               | Key không trùng | Có        | $O(\log n)$ |
| unordered_set/map | tùy             | Không     | $O(1)$ TB   |

Khi nào dùng set / map?

- Cần kiểm tra tồn tại nhanh
- Cần `lower_bound` / `upper_bound` (tìm "gần nhất")
- Cần phần tử min/max trong tập đang biến đổi (`insert/erase`)
- Đếm tần suất với key bất kỳ (string, pair, ...)

Hãy luyện thật nhiều – set / map sẽ trở thành công cụ mặc định trong tay bạn!